

Elaborato di Calcolo Numerico



Antonio Mascani

Matricola:7118851

E-mail: Antonio.mascani@edu.unifi.it

Andrea Pucci

Matricola:7119077

E-mail: Andrea.pucci4@edu.unifi.it

Anno Accademico 2024/2025

Esercizio 1. Combinare linearmente $y(t+h)$, $y(t)$, $y(t-h)$, $y(t-2h)$, $y(t-3h)$, in modo da ottenere una approssimazione di $y'(t)$ con errore di troncamento $O(h^4)$. Dettagliare il procedimento.

Cominciamo scrivendo l'espansione di Taylor dei termini:

$$y(t+h) = y(t) + hy'(t) + \frac{h^2}{2}y^{(2)}(t) + \frac{h^3}{6}y^{(3)}(t) + \frac{h^4}{24}y^{(4)}(t) + O(h^5)$$

$$y(t-h) = y(t) - hy'(t) + \frac{h^2}{2}y^{(2)}(t) - \frac{h^3}{6}y^{(3)}(t) + \frac{h^4}{24}y^{(4)}(t) + O(h^5)$$

$$y(t-3h) = y(t) - 3hy'(t) + \frac{9h^2}{2}y^{(2)}(t) - \frac{9h^3}{3}y^{(3)}(t) + \frac{27h^4}{8}y^{(4)}(t) + O(h^5)$$

$$y(t-2h) = y(t) - 2hy'(t) + 2h^2y^{(2)}(t) - \frac{4h^3}{3}y^{(3)}(t) + \frac{2h^4}{3}y^{(4)}(t) + O(h^5)$$

$$y(t) = y(t)$$

Per la combinazione lineare dobbiamo trovare A, B, C, D e E tali che:

$$Ay(t+h) + By(t) + Cy(t-h) + Dy(t-2h) + Ey(t-3h) = y'(t) + O(h^4)$$

Usando l'espansione di Taylor si ottiene:

$$\begin{aligned} & A(y(t) + hy'(t) + \frac{h^2}{2}y^{(2)}(t) + \frac{h^3}{6}y^{(3)}(t) + \frac{h^4}{24}y^{(4)}(t) + O(h^5)) + By(t) + C((t) - hy'(t) + \frac{h^2}{2}y^{(2)}(t) \\ & \quad - \frac{h^3}{6}y^{(3)}(t) \\ & + \frac{h^4}{24}y^{(4)}(t) + O(h^5)) + D\left(y(t) - 2hy'(t) + 2h^2y^{(2)}(t) - \frac{4h^3}{3}y^{(3)}(t) + \frac{2h^4}{3}y^{(4)}(t) + O(h^5)\right) + \\ & E(y(t) - 3hy'(t) + \frac{9h^2}{2}y^{(2)}(t) - \frac{9h^3}{3}y^{(3)}(t) + \frac{27h^4}{8}y^{(4)}(t) + O(h^5)) \end{aligned}$$

Ora raccogliamo:

$$\begin{aligned} & (A+B+C+D+E)y(t) + (A-C-2D-3E)hy'(t) + \left(\frac{A}{2} + \frac{C}{2} + \frac{9E}{2} + 2D\right)h^2y^{(2)}(t) + \\ & + \left(\frac{A}{6} - \frac{C}{6} - \frac{4D}{3} - \frac{9E}{2}\right)h^3y^{(3)}(t) + \left(\frac{A}{24} + \frac{C}{24} + \frac{2D}{3} + \frac{27E}{8}\right)h^4y^{(4)}(t) + O(h^5) \end{aligned}$$

Che si risolve attraverso il seguente sistema lineare:

$$\begin{cases} A+B+C+D+E=0 \\ (A-C-2D-3E)h=1 \\ \left(\frac{A}{2} + \frac{C}{2} + 2D + \frac{9E}{2}\right)h^2=0 \\ \left(\frac{A}{6} - \frac{C}{6} - \frac{4D}{3} - \frac{9E}{2}\right)h^3=0 \\ \left(\frac{A}{24} + \frac{C}{24} + \frac{2D}{3} + \frac{27E}{8}\right)h^4=0 \end{cases} \implies \begin{cases} A = \frac{1}{4h} \\ B = \frac{5}{6h} \\ C = -\frac{3}{2h} \\ D = \frac{1}{2h} \\ E = -\frac{1}{12h} \end{cases}$$

Quindi:

$$\frac{1}{4h}y(t+h) + \frac{5}{6h}y(t) - \frac{3}{2h}y(t-h) + \frac{1}{2h}y(t-2h) - \frac{1}{12h}y(t-3h) = y'(t) + O(h^4)$$

Esecizio 2. La quadrupla precisione IEEE utilizza 128 bit (16 byte) per rappresentare un numero reale, di cui: 1 per il segno della mantissa; 112 per la parte frazionaria della mantissa (per numeri normalizzati); 15 per l'esponente. La rappresentazione è con arrotondamento (evidentemente, la base è 2). Sapendo che lo shift dell'esponente vale 16383, calcolare: la precisione di macchina; il più grande numero di macchina normalizzato; il più piccolo numero di macchina normalizzato positivo. Qual è la rappresentazione del floating di 1/3? Qual è il corrispondente errore assoluto di rappresentazione?

Precisione di macchina

Definisco m = mantissa, b = base, s = esponente, v = shift

Nel caso di arrotondamento la precisione di macchina u è data dalla formula

$$u = \frac{b^{1-m}}{2}$$

Nel nostro caso abbiamo b=2 e m=113 (il bit in più è implicito) quindi la precisione di macchina è:

$$u = \frac{2^{1-113}}{2} = 2^{-113}$$

Numero di macchina normalizzato più grande e più piccolo

Il più grande numero di macchina normalizzato è dato dalla formula

$$r_2 = (1 - b^{-m})b^{\varphi}, \quad \varphi = b^s - v$$

Mentre il più piccolo numero di macchina normalizzato in valore assoluto è dato dalla formula

$$r_1 = b^{-v}$$

Nel nostro caso

$$r_2 = (1 - 2^{-113}) * 2^{2^{15} - 16383}$$

$$r_1 = 2^{-16383}$$

CHIEDERE A ALE PERCHE LUI FA +1 E -1 AGLI SHIFT

Rappresentazione in floating point di $\frac{1}{3}$

Per trovare la rappresentazione binaria di $\frac{1}{3}$ utilizzo il seguente algoritmo di conversione

$$2 * \frac{1}{3} = \frac{2}{3} = 0.\bar{6} \text{ la cui parte intera è } 0$$

$$2 * \frac{2}{3} = \frac{4}{3} \text{ la cui parte intera è } 1$$

$$2 * \frac{1}{3} = \frac{2}{3} = 0.\bar{6} \text{ la cui parte intera è } 0$$

Se si continuasse otterremmo una sequenza periodica di 0 e 1, di conseguenza abbiamo trovato che

$$\frac{1}{3} = 0.0\bar{1} \text{ in base } 2$$

Si pone il valore trovato in forma normalizzata e si ottiene

$$1.01010101... * 2^{-2}$$

Per trasformare l'esponente in base 2 per la quadrupla precisione IEEE 754 utilizziamo la formula

$$E = e + 2^{n-1} - 1 \text{ con } n = \text{numero di bit disponibili per l'esponente. Si ottiene quindi}$$

$$E = -2 + 16383 = 16381 \text{ che in base } 2 \text{ è } 01111111111101$$

Si mettono insieme i risultati ottenuti e abbiamo

$$fl(x) = 0 \quad 01111111111101 \quad 01010101...01$$

(segno, esponente, mantissa)

Errore assoluto di rappresentazione

Per calcolarlo si utilizza la formula

$$\text{Err} = |x - \text{fl}(x)| = |0.\bar{0}1 - 0.0101010101\dots0101| = \frac{1}{3} * 2^{-112}$$

Esercizio 3. Scrivere una function Matlab che implementi il metodo di Newton. Prevedere valori di default per la tolleranza di arresto e per il numero massimo di iterazioni.

```
1 function [x,valFunz] = newtonStandard(f,f1,x0,tol,itmax)
2
3 % [x,valFunz] = newtonStandard(f,f1,x0,tol,itmax)
4 %
5 % Input:
6 % f = funzione di partenza
7 % f1 = derivata prima della funzione f
8 % x0 = punto di partenza
9 % tol = tolleranza (default 1e-12)
10 % itmax = numero di iterazioni massime (default 1000)
11 %
12 % Output:
13 % x = approssimazione dello zero della funzione
14 % valFunz = valutazioni funzionali svolte durante l'esecuzione
15 % del codice
16 %
17 % Rel. 2-11-2025
18
19 if nargin < 5
20     itmax = 1000;
21 elseif itmax<1
22     error('maxit errato');
23 end
24
25 if nargin < 4
26     tol = 1e-12;
27 elseif tol < 0
28     error('tolleranza negativa');
29 end
30
31 if nargin < 3
32     error('numero argomenti di ingresso errato');
33 end
34
35 for i = 1:itmax
36     f1x = f1(x0);
37     if f1x==0
38         error('il metodo non converge');
39     end
40     fx = f(x0);
41     x = x0-fx/f1x;
42     err = abs(x-x0);
43     if err <= tol
44         break;
45     end
46     x0 = x;
47 end
48 if err > tol
49     warning('tolleranza richiesta non soddisfatta');
50 end
51 valFunz= i*2;
52 return
```

Esercizio 4. Scrivere una function Matlab che implementi il metodo di Newton modificato. Prevedere valori di default per la tolleranza di arresto e per il numero massimo di iterazioni.

```
1 % In input la funzione newtonModificato riceve:
2 % f = funzione passata
3 % x0 = punto di partenza scelto in modo arbitrario
4 % m = molteplicità funzione di partenza
5 % df = derivata funzione di partenza
6 % massima_iterazione = iterazioni massime che possono essere fatte
7 % tol = tolleranza scelta
8 %
9 % La funzione restituisce:
10 % x = approssimazione del risultato della funzione
11 % iter = numero di iterazioni impiegate
12
13 function [x, iterazione] = newtonModificato(f, x0, m, df, massima_iterazione, tol)
14     if nargin < 5
15         massima_iterazione = 100;
16     end
17     if nargin < 6
18         tol = 1e-11;
19     end
20
21     x = x0;
22     controlloTol = 0;
23
24     for iterazione = 1:massima_iterazione
25         fx = f(x);
26         der = df(x);
27
28         if der == 0 && fx == 0
29             break;
30         end
31         if der == 0
32             error('Non avviene convergenza (derivata nulla)');
33         end
34
35         nuovox = x - m * (fx / der);
36         controlloTol = abs(nuovox - x);
37
38         if controlloTol <= tol
39             x = nuovox;
40             return;
41         end
42         x = nuovox;
43     end
44
45     if controlloTol > tol
46         warning('Non è stata soddisfatta la tolleranza');
47     end
48 end
49
```

Esercizio 5. Scrivere una function Matlab che implementi il metodo di bisezione. Prevedere valori di default per la tolleranza di arresto.

```

1 function [x,valFunz]= Bisezione(f,a,b,tol)
2
3 % bisezione: metodo della bisezione per trovare uno zero di f(x) in [a, b]
4 %
5 % Input:
6 % f = funzione anonima
7 % a,b = estremi dell'intervallo iniziale [a, b], con f(a)*f(b) < 0
8 % tol      - (opzionale) tolleranza (default: 1e-12)
9 %
10 % Output:
11 % x = approssimazione dello zero
12 % valutazioniFunz = numero di iterazioni effettuate
13 %
14 %                                     Rel. 3-11-2025
15
16
17 if nargin == 3
18     tol =1e-12;
19 elseif nargin<3
20     error('dati in input non presenti');
21 end
22
23 if b<= a
24     error('intervallo iniziale errato');
25 end
26
27 if tol <= 0
28     error ('tolleranza errata');
29 end
30
31 fa = f(a);
32 valFunz=1;
33 if fa == 0
34     x=a;
35     return;
36 end
37
38 fb = f(b);
39 valFunz=valFunz+1;
40 if fb == 0
41     x=b;
42     return;
43 end
44 if fa*fb > 0
45     error('metodo non applicabile');
46 end
47
48 ..

```

```

47     maxit =ceil(-log2(tol / (b- a)));
48
49     for it = 1:maxit
50         x=(a+b)/2;
51         fx = f(x);
52         f1 = abs(fb-fa)/(b-a);
53         if abs(fx) <= f1*tol
54             break;
55
56         elseif fa*fx<0
57             b=x;
58             fb=fx;
59         else
60             a=x;
61             fa=fx;

```

```

62         end
63
64     end
65     valFunz=valFunz + it;
66     return
67 end
68
69

```

Esercizio 6. Tabulare il costo computazionale dei precedenti metodi, in termini di valutazioni di funzione, per determinare lo zero delle funzioni:

$$f(x) = x - \cos(x), \quad g(x) = f(x)^3,$$

considerando tolleranze: $10^{-3}, 10^{-6}, 10^{-9}, 10^{-12}$. Utilizzare $x_0 = 0$ per i metodi di Newton, e intervallo iniziale $[0, 1]$ per il metodo di bisezione. Commentare i risultati ottenuti.

Le seguenti due tabelle presentano i risultati per la funzione f e g della function *newtonStadard* con $x_0 = 0$:

Funzione f		
Tolleranza	Valutazioni Funzioni	Risultato
10^{-3}	8	0.7391
10^{-6}	10	0.7391
10^{-9}	10	0.7391
10^{-12}	12	0.7931

Funzione g		
Tolleranza	Valutazioni Funzioni	Risultato
10^{-3}	28	0.7373
10^{-6}	62	0.7391
10^{-9}	96	0.7391
10^{-12}	130	0.7931

Le seguenti due tabelle presentano i risultati per la funzione f e g della funzione *newtonModificatoMolteplicita* con, rispettivamente, molteplicità 1 e 3 e $x_0 = 0$

Funzione f		
Tolleranza	Valutazioni Funzioni	Risultato
10^{-3}	8	0.7391
10^{-6}	10	0.7391
10^{-9}	10	0.7391
10^{-12}	12	0.7931

Funzione g		
Tolleranza	Valutazioni Funzioni	Risultato
10^{-3}	8	0.7373
10^{-6}	10	0.7391
10^{-9}	10	0.7391
10^{-12}	-	Tol non soddisfatta

Nelle seguenti due tabelle vengono presentati i risultati per la funzione f e g della funzione *Bisezione*, con $x_0 = 0$ e intervallo $[0,1]$:

Funzione f		
Tolleranza	Valutazioni Funzioni	Risultato
10^{-3}	10	0.7383
10^{-6}	21	0.7391
10^{-9}	30	0.7391
10^{-12}	41	0.7931

Funzione g		
Tolleranza	Valutazioni Funzioni	Risultato
10^{-3}	4	0.7500
10^{-6}	16	0.7391
10^{-9}	22	0.7391
10^{-12}	32	0.7931

Come possiamo vedere nelle prime due tabelle, il metodo di Newton standard mantiene un numero di valutazioni funzionali stabile a mano a mano che diminuisce la tolleranza per la funzione f , per la funzione g invece, di molteplicità di radice di 3, il numero di valutazioni funzionali crescono velocemente. Ciò è dovuto al fatto che il metodo in questione ha una convergenza quadratica, mentre questa viene persa nel caso della funzione g , questo porta alla crescita velocissima del numero di iterazioni.

Possiamo poi osservare che il metodo di Newton modificato rimane con un numero di iterazioni stabile, questo perché la convergenza quadratica del metodo rimane tale.

Nell'ultimo caso poi, si osserva come il metodo di bisezione rimanga costante sia per entrambe le funzioni, con un numero di valutazioni funzionali che cresce con la precisione richiesta. Si osserva che la crescita rimane costante in accordo con la proprietà di convergenza lineare di questo metodo.

Per tutti si osservi che i metodi convergono a risultati molto vicini tra loro, all'aumentare della precisione la radice trovata è per tutti la stessa, per entrambe le funzioni.

Esercizio 7. Scrivere una function Matlab, con sintassi `x = trilow(L,b)`, che risolve un sistema triangolare inferiore $n \times n$, in cui la matrice dei coefficienti L è passata come un vettore di lunghezza $n(n+1)/2$.

```
1 function x = trilows(L,b)
2 % x = trilow(L,b)
3 %
4 % Input:
5 % L = la matrice dei coefficienti L è passata
6 % come un vettore di lunghezza n(n + 1)/2.
7 %
8 % b= vettore dei termini noti
9 %
10 % Output: soluzione del sistema
11 %
12
13 lung= length(b);
14 if lung*(lung+1)/2 ~= length(L)
15     error ("Dati errati");
16 end
17
18 m=0;
19 L=L(:);
20 b=b(:);
21
22 for i=1:lung
23     if L(i+m)==0
24         error ("Matrice singolare");
25     end
26     b(i) = b(i)- L(m+1:m+i-1)'*b(1:i-1);
27     b(i) = b(i) / L(m+i);
28     m=m+i;
29 end
30
31 x=b;
32 return
33 end
34
35
36
37
38
```

Rel. 2-11-2025

Esercizio 8. Scrivere una function che implementi efficientemente il metodo di fattorizzazione LU con pivoting parziale. Validare la function su tutti i possibili casi previsti nel codice.

```
1 function [A, perm] = LU_pivoting_parziale(A)
2
3     % Esegue la fattorizzazione LU di A utilizzando pivoting
4     % parziale (scambio di righe).
5     %
6     % Input:
7     %   A = matrice quadrata e non singolare
8     %
9     % Output:
10    %   A   = matrice contenente L e U sovrapposte
11    %   perm = vettore di permutazione delle righe
12
13
14    [rows, cols] = size(A);
15
16    if rows ~= cols
17        error('Errore: la matrice deve essere quadrata.');
```

-Matrice non quadrata

```
>> A = [1 3; 1 9; 9 2]
```

```
[A, p] = LU_pivoting_parziale(A)
```

```
A =
```

```
1     3
1     9
9     2
```

```
Error using LU_pivoting_parziale (line 17)
```

```
Errore: la matrice deve essere quadrata.
```

-Matrice singolare

```
>> A = [1 3; 2 6]

[A, p] = LU_pivoting_parziale(A)

A =

     1     3
     2     6

Error using LU_pivoting_parziale (line 26)
Errore: la matrice risulta singolare.
```

-Matrice non singolare e quadrata

```
>> A = [1 3; 2 5]

[A, p] = LU_pivoting_parziale(A)

A =

     1     3
     2     5

A =

     2.0000     5.0000
     0.5000     0.5000

p =

     2
     1
```

Esercizio 9. Utilizzare la precedente function per risolvere i seguenti sistemi lineari di dimensione n ,

$$\begin{pmatrix} 1^0 & 2^0 & \dots & n^0 \\ 1^1 & 2^1 & \dots & n^1 \\ \vdots & \vdots & & \vdots \\ 1^{n-1} & 2^{n-1} & \dots & n^{n-1} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} \sum_{i=1}^n i^0 \\ \sum_{i=1}^n i^1 \\ \vdots \\ \sum_{i=1}^n i^{n-1} \end{pmatrix},$$

la cui soluzione è $x_i = 1$, $i = 1, \dots, n$, per $n = 1, \dots, 20$. Tabulare *convenientemente* l'errore, e spiegare i risultati ottenuti.

```

1 function x = LUriisolve(LU, x)
2
3 % Risolve il sistema lineare LU * x = b, con pivoting parziale.
4 %
5 % Input:
6 % LU = matrice quadrata contenente i coefficienti del sistema
7 % x = vettore dei termini noti
8 %
9 % Output:
10 % x = soluzione del sistema lineare
11
12 if nargin < 2
13     error('Errore: numero di argomenti insufficiente. Fornire LU e il vettore b.');
```

```

14 end
15
16 [rows, cols] = size(LU);
17 b_len = length(x);
18 if rows ~= cols || b_len ~= rows
19     error('Errore: dimensioni non compatibili. Controllare matrice e vettore.');
```

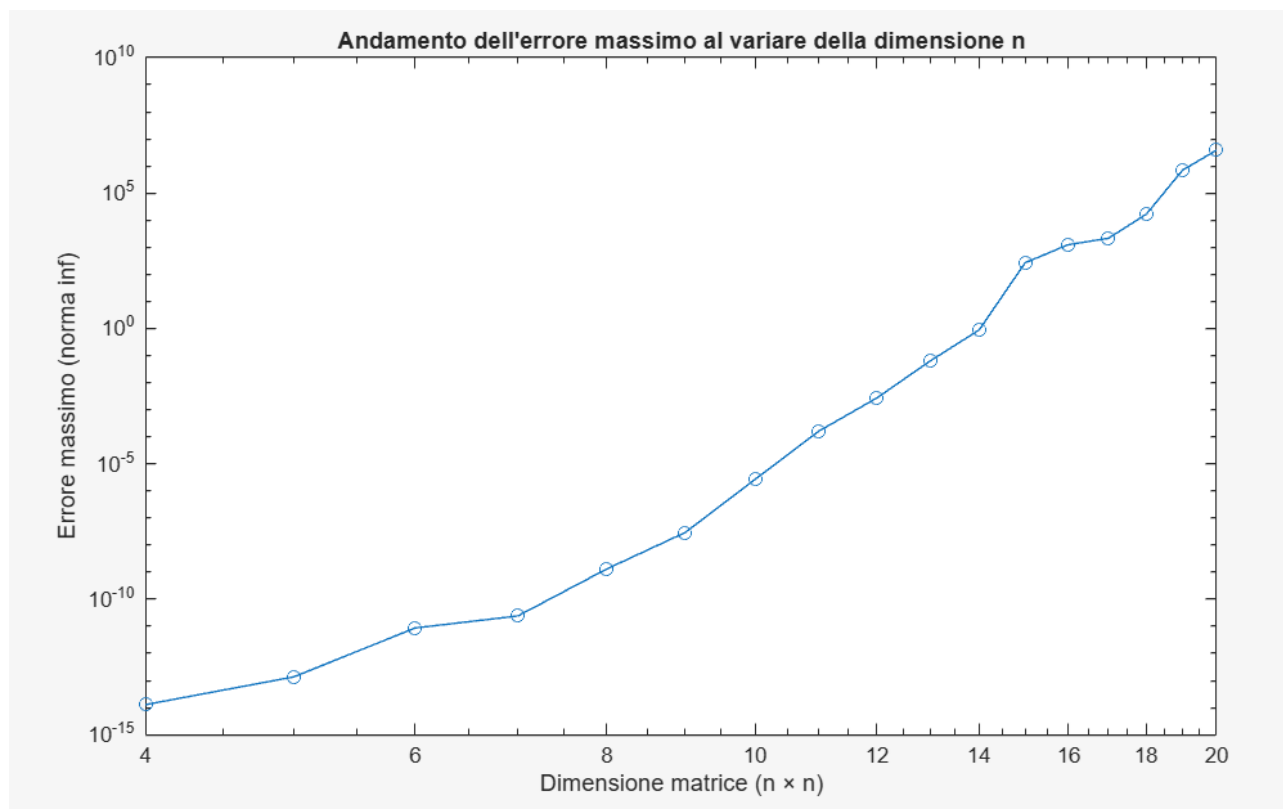
```

20 end
21 [LU, perm] = LU_pivoting_parziale(LU);
22 x = x(:);
23 x = x(perm);
24 for i = 1:rows
25     x(i+1:rows) = x(i+1:rows) - LU(i+1:rows, i) * x(i);
26 end
27
28 for i = rows:-1:1
29     if LU(i,i) == 0
30         error('Errore: fattore U singolare, impossibile proseguire.');
```

```

31     end
32     x(i) = x(i) / LU(i,i);
33     x(1:i-1) = x(1:i-1) - LU(1:i-1, i) * x(i);
34 end
35
36 end
37
```

La funzione calcola l'errore per le matrici che arrivano fino alla dimensione $n \times n$ e per matrici piccole l'errore risulta trascurabile. Più le matrici crescono di dimensione e meno l'errore diventa trascurabile, fino a raggiungere addirittura un errore dell'ordine di 10^6 . Questi risultati derivano dal fatto che la matrice utilizzata per il test è la matrice di Vandermonde ossia una matrice mal condizionata.



ITER	MAX ERRORE
1	0,0000000000000000
2	0,0000000000000000
3	0,0000000000000000
4	0,0000000000000012
5	0,0000000000000134
6	0,0000000000008632
7	0,000000000023878
8	0,000000001298100
9	0,000000028699000
10	0,000002666200000
11	0,000158190000000
12	0,002634400000000
13	0,061293000000000
14	0,872440000000000
15	263,4000000000000
16	1221,2000000000000
17	2116,1000000000000
18	16589000000000000
19	688490,00000000000
20	3801200,0000000000

Esercizio 10. Scrivere una function che implementi efficientemente il metodo di fattorizzazione LDL^T per matrici simmetriche e definite positive. Validare la function su tutti i possibili casi previsti nel codice.

```

1 %
2 % A = ldl_fact(A)
3 %
4 % Input:
5 % A = matrice non singolare simmetrica definita positiva (sdp)
6 %
7 % Output:
8 % A = matrice fattorizzata  $LDL^T$ 
9 %
10 %
11 %                               Rel. 2-11-2025
12
13 function A = ldl_fattorizzazione(A)
14     [m,n] = size(A);
15     if m ~= n
16         error('matrice non quadrata');
17     end
18     if A(1,1) <= 0
19         error('A non sdp');
20     end
21
22     A(2:n,1) = A(2:n,1) / A(1,1);
23
24     for j = 2:n
25         v = A(j,1:j-1)' .* diag(A(1:j-1,1:j-1));
26         A(j,j) = A(j,j) - A(j,1:j-1) * v;
27         if A(j,j) <= 0
28             error('A non semidefinita positiva');
29         end
30         A(j+1:n,j) = (A(j+1:n,j) - A(j+1:n, 1:j-1)*v) / A(j,j);
31     end
32     return
33 end

```

Matrice non quadrata

```
>> A = [3 5 9 ; 9 8 7]
ldl_fattorizzazione(A)
```

A =

```

3     5     9
9     8     7

```

Error using [ldl_fattorizzazione](#) (line 16)
matrice non quadrata

Matrice non sdp

```
>> A = [1 2 3 ; 0 -1 4; 2 1 1]  
ldl_fattorizzazione(A)
```

A =

1	2	3
0	-1	4
2	1	1

Error using [ldl_fattorizzazione](#) ([line 28](#))
A non semidefinita positiva

Matrice quadrata e sdp

```
>> A = [4 1 ; 1 3]  
ldl_fattorizzazione(A)
```

A =

4	1
1	3

ans =

4.0000	1.0000
0.2500	2.7500

Esercizio 11. Utilizzare la precedente function per risolvere i seguenti sistemi lineari di dimensione n ,

$$\begin{pmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \cdots & \frac{1}{n} \\ \frac{1}{2} & \frac{1}{3} & & \ddots & \vdots \\ \frac{1}{3} & & \frac{1}{n} & & \vdots \\ \vdots & \ddots & & \ddots & \frac{1}{2n-2} \\ \frac{1}{n} & \cdots & \cdots & \frac{1}{2n-2} & \frac{1}{2n-1} \end{pmatrix} \begin{pmatrix} x_1 \\ \vdots \\ \vdots \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} \sum_{i=1}^n i^{-1} \\ \sum_{i=2}^{n+1} i^{-1} \\ \vdots \\ \sum_{i=n}^{2n-1} i^{-1} \end{pmatrix},$$

la cui soluzione è $x_i = 1$, $i = 1, \dots, n$, per $n = 1, \dots, 20$. Tabulare *convenientemente* l'errore, e spiegare i risultati ottenuti.

```

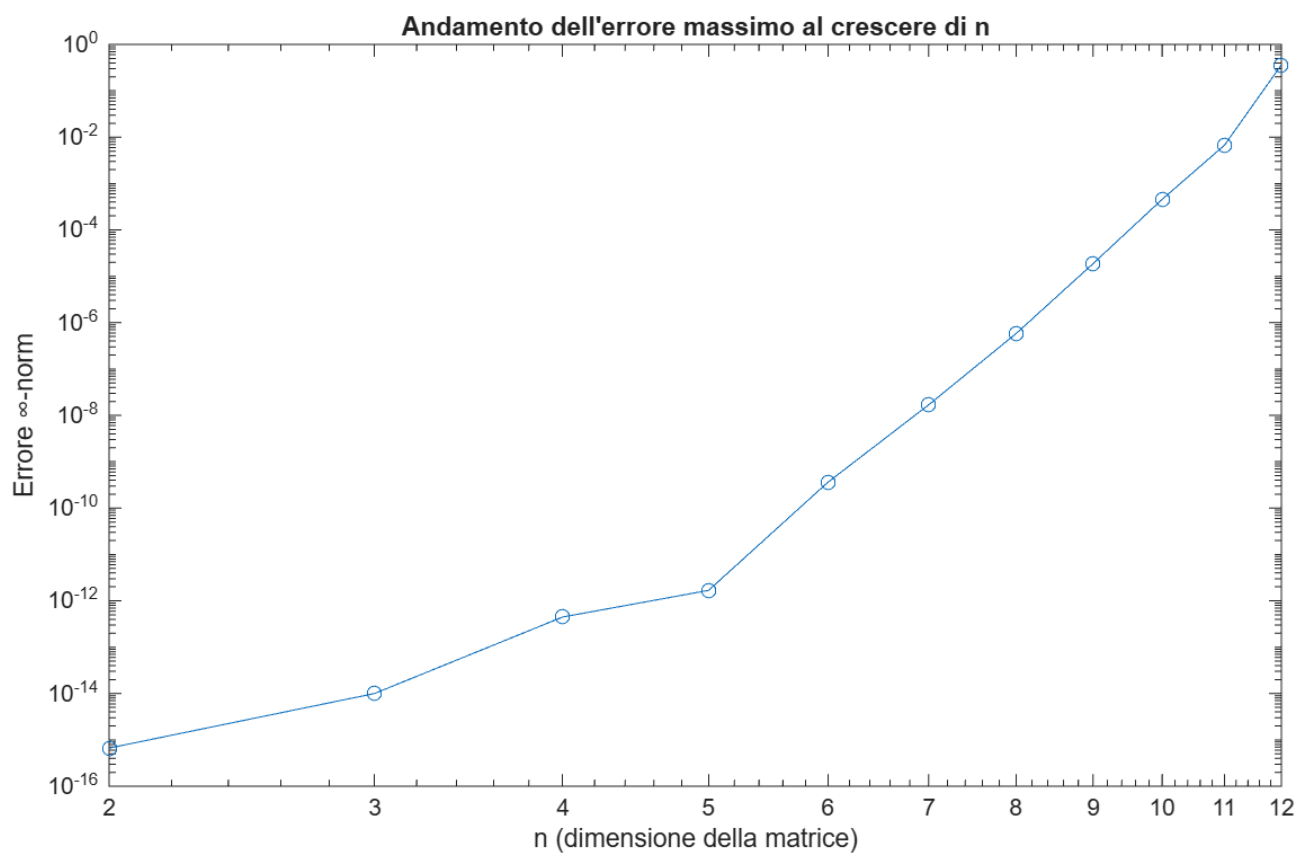
1 % Questa funzione utilizza la fattorizzazione LDL per risolvere il sistema
2 % Ax = b.
3 % Output:
4 % s: vettore soluzione del sistema lineare
5 % Input:
6 % A_matrice: matrice dei coefficienti
7 % s_rhs_vettore: Vettore dei termini noti
8 % 2-11_2025
9 function s = LDL(A_matrice, s_rhs_vettore)
10
11     if nargin < 2
12         error('gli argomenti da inserire sono 2');
13     end
14     [n, m] = size(A_matrice);
15
16
17     if n ~= m || m ~= length(s_rhs_vettore)
18         error(['Dimensioni non valide: la matrice dev\'essere quadrata e' ...
19             ' la lunghezza delle righe e delle colonne deve corrispondere alla lunghezza di b']);
20     end
21
22     A_fattorizzata = ldl_fattorizzazione(A_matrice);
23     vettore_soluzione = s_rhs_vettore(:);
24     for i = 1:n
25         vettore_soluzione(i+1:n) = vettore_soluzione(i+1:n) - A_fattorizzata(i+1:n, i) *
vettore_soluzione(i);
26     end
27     vettore_soluzione = vettore_soluzione ./ diag(A_fattorizzata);
28
29     for i = n:-1:1
30         vettore_soluzione(1:i-1) = vettore_soluzione(1:i-1) - A_fattorizzata(i, 1:i-1)' *
vettore_soluzione(i);
31     end
32     s = vettore_soluzione;
33 end

```

La matrice in questione è la matrice di Hilbert che dalla dimensione uguale a 13 in poi risulta non essere più sdp. I test sono stati quindi eseguiti solo fino alla matrice quadrata di dimensioni 12.

Iter	Max errore
1	0.0000000000000000
2	0.0000000000000001
3	0.0000000000000010
4	0.0000000000000447
5	0.000000000001680

6	0.000000000360474
7	0.000000016933495
8	0.000000581792409
9	0.000018829668825
10	0.000455475202895
11	0.006712415980578
12	0.363668019548027



Esercizio 12. Scrivere una function Matlab che implementi efficientemente la fattorizzazione QR di una matrice di rango massimo per risolvere un sistema lineare sovradeterminato $A\mathbf{x} = \mathbf{b}$ nel senso dei minimi quadrati. Ritornare, in uscita, la norma del relativo residuo, $\|\mathbf{r}\|_2^2$. Validare la function su tutti i possibili casi previsti nel codice.

```

1 % Esegue la fattorizzazione QR di una matrice A utilizzando
2 % le trasformazioni di Householder (metodo implicito).
3 %
4 % Input:
5 %   A = matrice di dimensione m x n, con m > n e rango(A) = n
6 %
7 % Output:
8 %   A = matrice contenente la fattorizzazione QR
9 %
10                                     2-11-2025
11 function A = qr_fattorizza(A)
12     [m, n] = size(A);
13     if m < n
14         error('La matrice deve avere più righe che colonne (sistema sovradeterminato).');
15     end
16
17     for i = 1:n
18         alfa = norm(A(i:m, i));
19         if alfa == 0
20             error('La matrice non è a rango pieno: impossibile proseguire la fattorizzazione.');

```

Codice matlab che restituisce la norma del relativo residuo in uscita $\|r\|_2^2$

```
1 function A = qr_decomp(A)
2
3 % A = qr_decomp(A)
4 %
5 % Input:
6 % A = matrice mxn con m>n = rank(A)
7 %
8 % Output:
9 % A = matrice fattorizzata QR
10 %
11
12
13 [m, n] = size(A);
14 if m < n
15     error('Il sistema deve essere sovradeterminato');
16 end
17
18 for i = 1:n
19     alfa = norm(A(i:m,i));
20     if alfa == 0
21         error('La matrice non ha rango massimo');
22     end
23     if A(i,i) > 0
24         alfa = -alfa;
25     end
26     v1 = A(i,i) - alfa;
27     A(i,i) = alfa;
28     A(i+1:m,i) = A(i+1:m,i) / v1;
29     beta = -v1 / alfa;
30     A(i:m,i+1:n) = A(i:m,i+1:n) - (beta * [1; A(i+1:m,i)]) * [1, A(i+1:m,i)'] * A(i:m,i+1:n);
31 end
32 return;
33 end
```

sistema non sovradeterminato

```
>> A = [1 ,2, 3; 1,2,3]
```

A =

1	2	3
1	2	3

```
>> A = qr_decomp(A)
```

Error using `qr_decomp` (line 15)

Il sistema deve essere sovradeterminato

matrice non con rango massimo

A =

1	2	3
2	4	6
3	6	9
4	8	12
5	10	15

```
>> A = qr_decomp(A)
```

Error using `qr_decomp` (line 21)

La matrice non ha rango massimo

“dimensioni del vettore non corrispondono al numero di righe della matrice”

A =

1	2	3
4	5	6
7	8	10
1	0	0
0	1	0

>> b= [1;2]

b =

1
2

>> [x_solution, residual_norm_squared] = QR_least_squares_solve(A, b)

Error using [QR_least_squares_solve](#) (line 22)

Le dimensioni del vettore b non corrispondono al numero di righe della matrice A.

fattorizzazione QR

A =

1	2	3
4	5	6
7	8	10
1	0	0
0	1	0

>> A = qr_decomp(A)

A =

-8.1854	-9.5292	-11.8504
0.4355	1.7872	1.7204
0.7621	0.4349	1.2679
0.1089	0.6943	0.0600
0	-0.5531	0.7405

Esercizio 13. Modificare la precedente function in modo che minimizzi, indicato al solito $\mathbf{r} := \mathbf{Ax} - \mathbf{b} \in \mathbb{R}^m$ il residuo del sistema lineare sovradeterminato assegnato, invece che $\|\mathbf{r}\|_2^2 = \sum_{i=1}^m r_i^2$, la seguente norma pesata:

$$\|\mathbf{r}\|_\omega^2 = \sum_{i=1}^m \omega_i r_i^2, \quad (1)$$

dove $\omega_1, \dots, \omega_m$ sono pesi positivi assegnati.

```

1 function [x_soluzione, norma_residuo_quadrato_pesata] = QR_minimi_quadrati_pesata(matrice_A,
   vettore_b, vettore_pesi)
2
3 % Risolve il problema dei minimi quadrati pesati (WLS) per il sistema sovradeterminato A*x = b.
4 % L'obiettivo è minimizzare il quadrato della norma euclidea pesata: || D * (A*x - b) ||_2^2,
5 % dove D è la matrice diagonale dei fattori di peso (D = diag(sqrt(vettore_pesi))).
6 % La soluzione è calcolata tramite la fattorizzazione QR (metodo di Householder implicito).
7 %
8 % Input:
9 %   matrice_A: Matrice dei coefficienti m x n (m > n e rank(A) = n).
10 %   vettore_b: Vettore dei termini noti di dimensione m.
11 %   vettore_pesi: Vettore dei pesi (w_i) di dimensione m.
12 % Output:
13 %   x_soluzione: Vettore soluzione x di dimensione n.
14 %   norma_residuo_quadrato_pesata: Valore minimo di || D * (A*x - b) ||_2^2.
15 %                               2-11-2025
16 [num_righe, num_colonne] = size(matrice_A);
17 if length(vettore_b) ~= num_righe || length(vettore_pesi) ~= num_righe
18     error('Le dimensioni del vettore b o del vettore dei pesi non corrispondono al numero di
   righe (m) della matrice A.');
```

end

```

20
21 vettore_b_trasformato = vettore_b(:);
22 fattori_peso_radice = sqrt(vettore_pesi(:));
23 vettore_b_trasformato = fattori_peso_radice .* vettore_b_trasformato;
24 matrice_A_trasformata = fattori_peso_radice .* matrice_A;
25
26 matrice_R_e_riflessioni = qr_fattorizza(matrice_A_trasformata);
27
28 for i = 1:num_colonne
29     vettore_householder = [1; matrice_R_e_riflessioni(i+1:num_righe, i)];
30     fattore_beta = 2 / (vettore_householder' * vettore_householder);
31     vettore_b_trasformato(i:num_righe) = vettore_b_trasformato(i:num_righe) - fattore_beta *
   vettore_householder * (vettore_householder' * vettore_b_trasformato(i:num_righe));
32 end
33
34 parte_residua_b_tilde = vettore_b_trasformato(num_colonne+1:num_righe);
35 norma_residuo_quadrato_pesata = norm(parte_residua_b_tilde)^2;
36
37 for i = num_colonne:-1:1
38     vettore_b_trasformato(i) = vettore_b_trasformato(i) / matrice_R_e_riflessioni(i, i);
39     vettore_b_trasformato(1:i-1) = vettore_b_trasformato(1:i-1) - matrice_R_e_riflessioni(1:i-1,
   i) * vettore_b_trasformato(i);
40 end
41 x_soluzione = vettore_b_trasformato(1:num_colonne);
42
43 end
```

Esercizio 14. Siano assegnati:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 3 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}.$$

Risolvere il sistema $A\mathbf{x} = \mathbf{b}$ nel senso dei minimi quadrati, calcolando la soluzione e $\|\mathbf{r}\|_2^2$. Assegnati i pesi $\omega_i = 5 - i$, $i = 1, \dots, 4$, calcolare la soluzione (e la relativa norma pesata), ottenuta minimizzando (I).

RISULTATO MINIMI QUADRATI STANDARD (QR_minimi_quadrati)

```
=====
x(1) = 4.4865
x(2) = -7.0270
x(3) = 3.2703
```

RISULTATO MINIMI QUADRATI PESATI (QR_minimi_quadrati_pesata)
(Pesi w = [4, 3, 2, 1])

```
=====
x(1) = 4.3396
x(2) = -6.9283
x(3) = 3.2943
```

Esercizio 15. Scrivere una function che implementi efficientemente il metodo di Newton per sistemi di equazioni nonlineari.

```
1 function root_approximation = newton_non_lineare(func_handle, jac_handle, start_vector, rel_tol,
2 max_iters)
3 % newton_non_lineare: Metodo di Newton per sistemi non lineari
4 % Input:
5 % func_handle : handle di un vettore di n funzioni non lineari
6 % jac_handle : handle della matrice Jacobiana n×n associata
7 % start_vector: vettore di partenza per l'iterazione
8 % rel_tol : tolleranza relativa richiesta
9 % max_iters : limite massimo di iterazioni consentite
10 % Output:
11 % root_approximation : stima finale della radice del sistema
12 % Rel. 02-11-2025
13
14 if nargin < 3
15     error('Numero insufficiente di argomenti in ingresso.');
```


Esercizio 16. Utilizzare la precedente function per calcolare il punto stazionario della funzione

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top Q \mathbf{x} - \sum_{i=1}^3 \sin(x_i), \quad Q = \begin{pmatrix} 4 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 4 \end{pmatrix},$$

a partire dal punto iniziale $\mathbf{x}_0 = \mathbf{0} \in \mathbb{R}^3$.

Per calcolare il punto stazionario di $f(\mathbf{x})$ bisogna trovare il punto in cui il gradiente di $f(\mathbf{x}) = 0$ ossia $F(\mathbf{x})$ vettore nullo.

Se considero il vettore $\mathbf{x} = (x, y, z)$ il vettore gradiente corrispondente è

$$F(\mathbf{x}) = \begin{pmatrix} 4x + y - \cos(x) \\ x + 3y + z - \cos(y) \\ y + 4z - \cos(z) \end{pmatrix}$$

La matrice Jacobiana corrispondente è:

$$J(\mathbf{x}) = \begin{pmatrix} 4 + \sin(x) & 1 & 0 \\ 1 & 3 + \sin(y) & 1 \\ 0 & 1 & 4 + \sin(z) \end{pmatrix}$$

Sostituisco i dati nel codice Matlab sviluppati precedentemente:

F =

[function handle](#) with value:

```
@(x)[4*x(1)+x(2)-cos(x(1));x(1)+3*x(2)+x(3)-cos(x(2));x(2)+4*x(3)-cos(x(3))]
```

J =

[function handle](#) with value:

```
@(x)[4+sin(x(1)),1,0;1,3+sin(x(2)),1;0,1,4+sin(x(3))]
```

Il risultato è il seguente:

```
>> approssimazione_radice = newton_non_lineare(F, J, x, 1e-10, 100)

approssimazione_radice =

    0.1962
    0.1962
    0.1962
```

Esercizio 17. Scrivere una function Matlab,

$$y = \text{lagrange}(xi, fi, x)$$

in cui (xi,fi) contengono i dati del problema e x è un vettore di punti a cui corrispondono le valutazioni del polinomio interpolante in forma di Lagrange restituite nel vettore y.

```
1 function y=lagrange(xi,fi,x)
2
3 %
4 % y=lagrange(xi,fi,x)
5 %
6 % Input:
7 % xi = ascisse di interpolazione
8 % fi = valore della funzione nelle ascisse di interpolazione
9 % x = vettore di punti
10 %
11 % Output:
12 % y = vettore contenente le valutazioni del polinomio interpolante
13 % Rel. 2-11-2025
14
15 if nargin < 3
16     error('Dati insufficienti');
17 end
18
19
20 grado_n = length(xi);
21 if grado_n ~= length(fi)
22     error('dati_inseriti_errati');
23 end
24 y = zeros(size(x));
25
26 for i = 1:grado_n
27     L = ones(size(x));
28     for j = 1:grado_n
29         if j ~= i
30             L = L .* (x - xi(j)) / (xi(i) - xi(j));
31         end
32     end
33     y = y + fi(i) * L;
34 end
35 end
36
```

Esercizio 18. Scrivere una function Matlab,

$$y = \text{newton}(xi, fi, x)$$

che ha la stessa sintassi e funzione della function del precedente esercizio, ma con l'utilizzo della forma di Newton del polinomio interpolante.

```
1 function valori = newton(punti_x, valori_f, x_valori)
2
3 % Newton polinomio interpolante
4 %
5 % Input:
6 % punti_x = vettore delle ascisse di interpolazione
7 % valori_f = valori della funzione corrispondenti alle ascisse
8 % x_valori = vettore dei punti in cui valutare il polinomio
9 %
10 % Output:
11 % valori = vettore dei valori del polinomio interpolante
12 % 2-11-2025
13 n = length(punti_x);
14 if length(valori_f) ~= n
15     error('I vettori punti_x e valori_f devono avere la stessa lunghezza.');
```

```
16 end
17
18 % Costruzione della tabella delle differenze divise
19 tabella_divise = zeros(n, n);
20 tabella_divise(:, 1) = valori_f(:);
21 for col = 2:n
22     for riga = 1:n - col + 1
23         tabella_divise(riga, col) = ...
24             (tabella_divise(riga+1, col-1) - tabella_divise(riga, col-1)) / ...
25             (punti_x(riga+col-1) - punti_x(riga));
26     end
27 end
28
29 % Coefficienti del polinomio di Newton
30 coeff_newton = tabella_divise(1, :);
31
32 % Valutazione del polinomio nei punti richiesti
33 valori = zeros(size(x_valori));
34 for k = 1:length(x_valori)
35     x_corrente = x_valori(k);
36     termine = coeff_newton(1);
37     prodotto = 1;
38     for j = 2:n
39         prodotto = prodotto * (x_corrente - punti_x(j-1));
40         termine = termine + coeff_newton(j) * prodotto;
41     end
42     valori(k) = termine;
43 end
44 end
45
```

Esercizio 19. Scrivere una function Matlab,

$[y, y1] = \text{hermite}(xi, fi, fi1, x)$

in cui $(xi, fi, fi1)$ contengono i dati del problema e x è un vettore di punti a cui corrispondono le valutazioni del polinomio interpolante di Hermite, e della sua derivata prima, rispettivamente restituite nei vettori y e $y1$.

```
1 function [y, y1] = hermite(xi, fi, fi1, x)
2 % [y, y1] = hermite(xi, fi, fi1, x)
3 %
4 % Input:
5 % xi : Ascisse di interpolazione.
6 % fi : Valori della funzione nelle ascisse di interpolazione (f(xi)).
7 % fi1 : Valori della derivata della funzione nelle ascisse di interpolazione (f'(xi)).
8 % x : Vettore di punti in cui valutare il polinomio.
9 %
10 % Output:
11 % y : Valutazioni del polinomio di Hermite nei punti x.
12 % y1 : Valutazioni del polinomio di Hermite derivato nei punti x.
13 %
14 % Rel. 2-11-2025
15
16 % Controllo dei dati messi in input
17 if nargin < 4
18     error('hermite:datiInsufficienti', 'Errore: Dati insufficienti. Servono almeno 4
    argomenti. ');
19 end
20
21 n = length(xi) - 1;
22 if n ~= length(fi) - 1
23     error('hermite:datiErrati', 'Errore: Le lunghezze di xi e fi errate. ');
24 end
25
26
27 f_estesa = zeros(1, 2 * length(fi));
28 f_estesa(1:2:end) = fi;
29 f_estesa(2:2:end) = fi1;
30
31 % Duplica le ascisse di interpolazione per le differenze divise
32 xi_estesa = zeros(1, 2 * length(xi));
33 xi_estesa(1:2:end) = xi;
34 xi_estesa(2:2:end) = xi;
35
36 % Calcola le differenze divise di Hermite
37 differenza_divise = calcoloHermiteDifferenzaDivise(xi_estesa, f_estesa);
38
39
40 y = differenza_divise(2*n+2) * ones(size(x));
41 y1 = zeros(size(x));
42
43
44 for i = (2*n+1):-1:1
45     y1 = y + (x - xi_estesa(i)) .* y1;
46     y = y .* (x - xi_estesa(i)) + differenza_divise(i);
47 end
48
49 end
```

```

50
51 % --- Sottofunzione per il calcolo delle differenze divise di Hermite ---
52 function fi_dd = calcoloHermiteDifferenzaDivise(xi_estese, fi_estese)
53
54 %   fi_dd = calcoloHermiteDifferenzeDivise(xi_ext, fi_ext)
55 %
56 %   Input:
57 %       xi_estese : Ascisse estese per le differenze divise.
58 %       fi_estese : Valori della funzione e della derivata estesi.
59 %
60 %   Output:

```

```

61 %       fi_dd : Vettore delle differenze divise di Hermite.
62 %
63 %                                     2-11-2025
64
65 if nargin < 2
66     error('dati insufficienti per il calcolo delle differenze divise');
67 end
68
69 numero_punti = length(xi_estese);
70 fi_dd = fi_estese;
71
72
73 for i = (numero_punti - 1):-2:3
74     fi_dd(i) = (fi_dd(i) - fi_dd(i-2)) / (xi_estese(i) - xi_estese(i-1));
75 end
76
77
78 for j = 2:(numero_punti - 1)
79     for i = numero_punti:-1:(j + 1)
80         fi_dd(i) = (fi_dd(i) - fi_dd(i-1)) / (xi_estese(i) - xi_estese(i-j));
81     end
82 end
83
84 end

```

Esercizio 20. Scrivere una function Matlab,

$x = \text{chebyshev}(a, b, n)$

che restituisca le ascisse di Chebyshev per il polinomio interpolante di grado n sull'intervallo $[a, b]$.

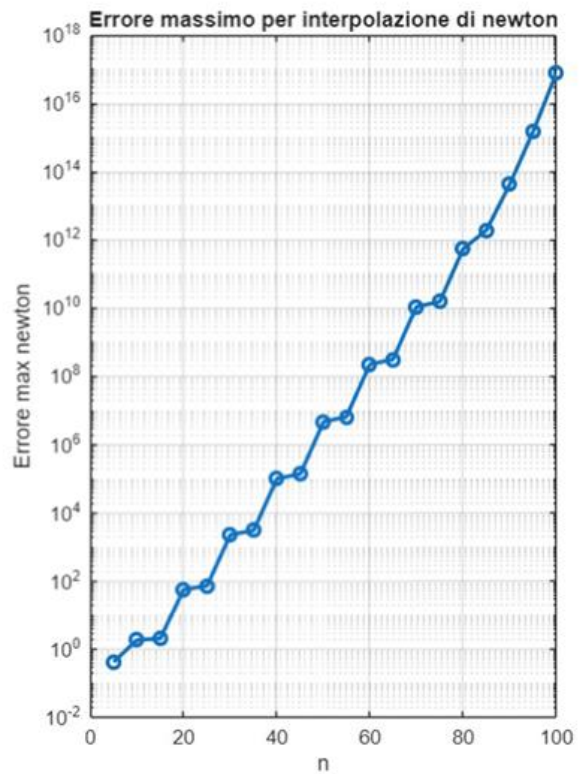
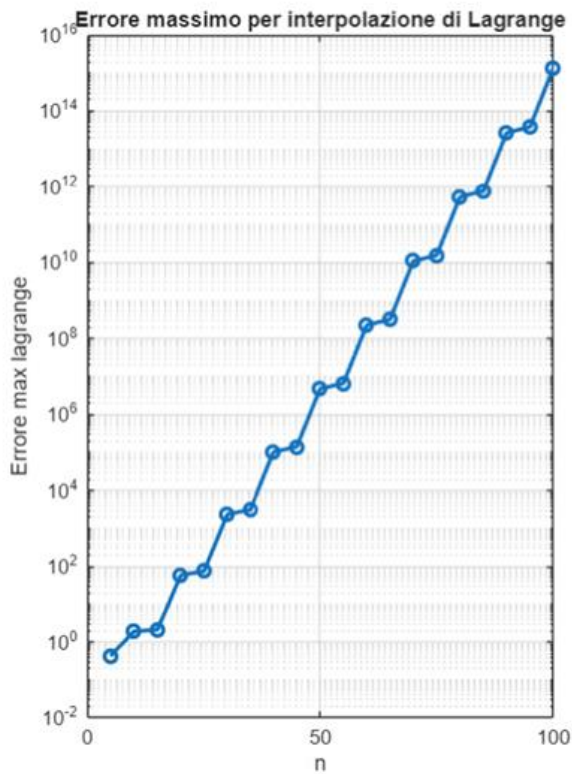
```
1 % x = CHEBYSHEV(a, b, n)
2 %
3 % Input :
4 %   a,b = estremi dell'intervallo [a, b]
5 %   n   = grado del polinomio (numero di suddivisioni - 1)
6 %
7 % Output:
8 %   x   = nodi di Chebyshev nell'intervallo [a, b]
9 %                                           2-11-2025
10 function x = chebyshev(a, b, n)
11
12     if n < 0
13         error('Il parametro n deve essere un intero >= 0');
14     end
15
16     x_cheb_std = zeros(n + 1, 1);
17     x = zeros(n + 1, 1);
18
19     for k = 0:n
20         x_cheb_std(k + 1) = cos(((2 * k + 1) * pi) / (2 * (n + 1)));
21     end
22
23     for k = 0:n
24         x(k + 1) = ((a + b) / 2) + ((b - a) / 2) * x_cheb_std(k + 1);
25     end
26
27 end
28
```

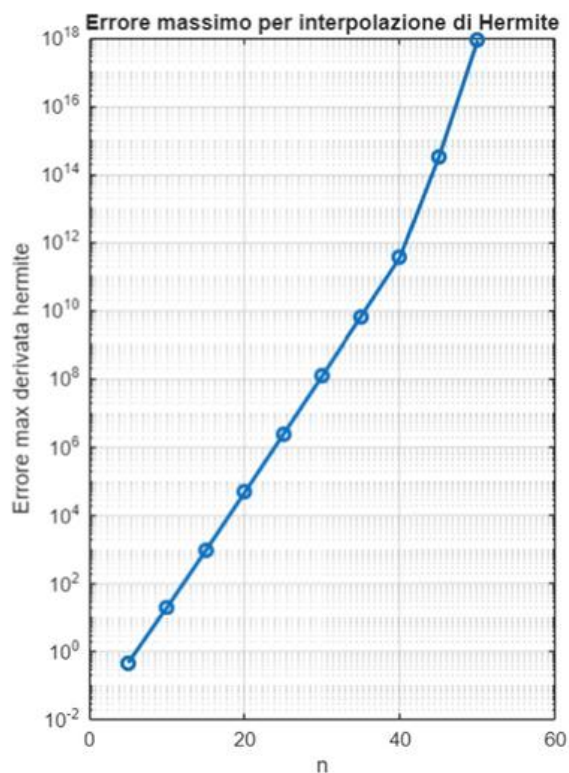
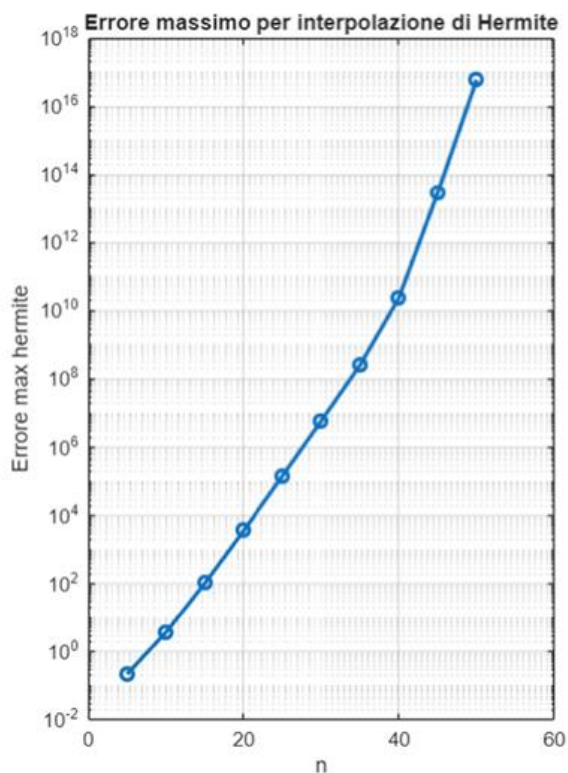
Esercizio 21. Graficare l'errore massimo (calcolato come il massimo di 10001 punti equispaziati nell'intervallo) per i polinomi interpolanti degli esercizi 17-19, relative alla funzione di Runge,

$$f(x) = (1 + x^2)^{-1}, \quad x \in [-5, 5],$$

utilizzando $n + 1$ ascisse equispaziate, con $n = 5, 10, 15, \dots, 100$, per i polinomi interpolanti in 17-18, e $n = 5, 10, 15, \dots, 50$, per il polinomio di Hermite. Per quest'ultimo polinomio, graficare anche l'errore della derivata prima. Utilizzare il modo più appropriato per graficare i risultati. Commentare i risultati ottenuti.

Dai grafici sotto possiamo vedere come i dati crescano in modo elevato all'aumentare di n . Questo aumentare è causata dalla crescita esponenziale del numero di condizionamento con n (dovuto da una complessità circa di $O(2^n)$), il problema diventa quindi numericamente instabile. Il problema dell'interpolazione con ascisse equidistanti è, infatti, malcondizionato.



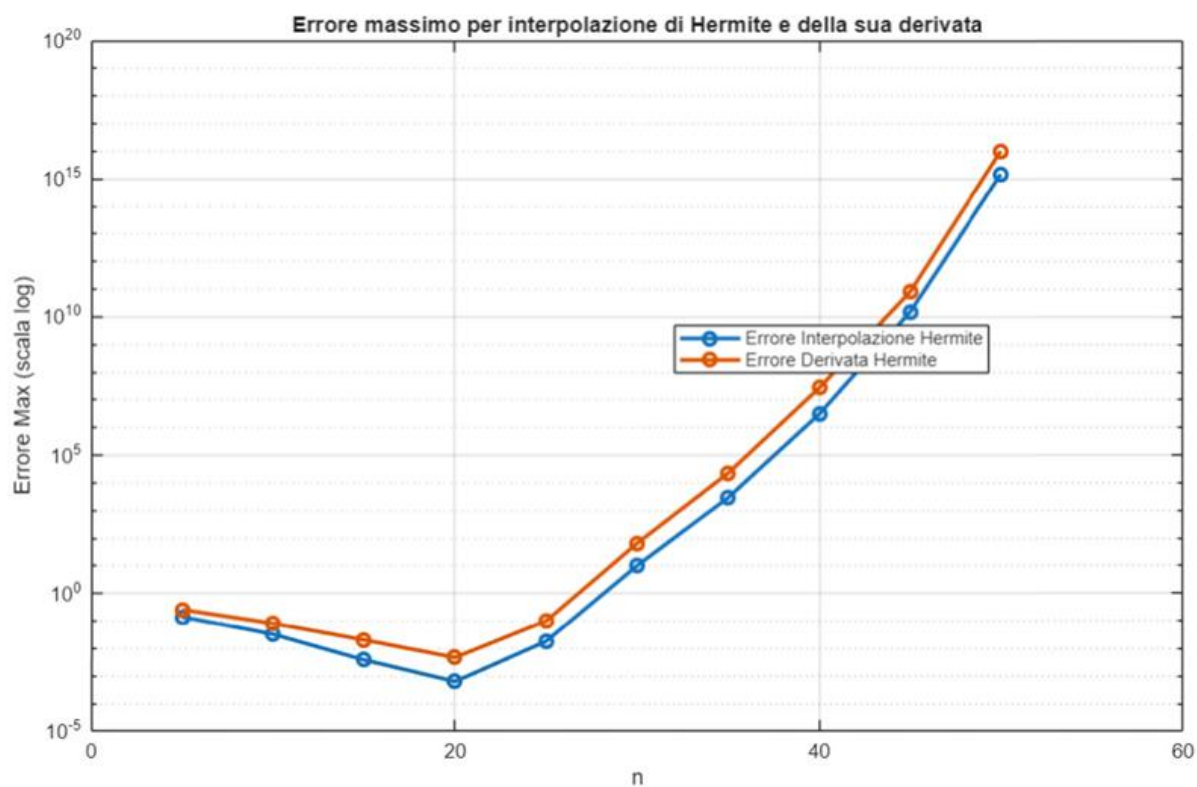
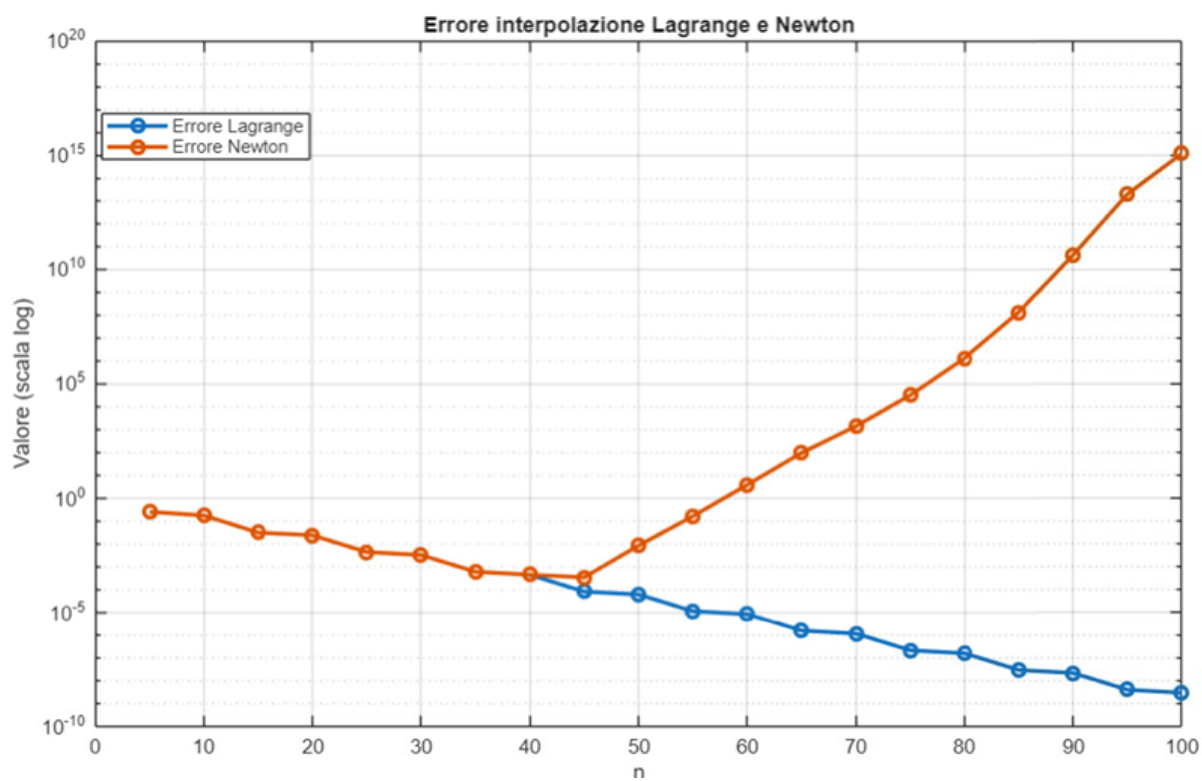


Esercizio 22. Ripetere l'esercizio precedente, utilizzando le ascisse di Chebyshev in luogo di quelle equidistanti.

Nella prima figura, la scelta delle **ascisse di Chebyshev** per la **forma di Lagrange** si dimostra corretta, in quanto riduce efficacemente l'**errore teorico** e attenua il malcondizionamento del problema.

Per la **forma di Newton**, invece, l'errore calcolato decresce fino a un certo n , dopodiché mostra una **ripida crescita** (divergenza). Questo comportamento illustra che, in **aritmetica finita**, le formulazioni, pur essendo equivalenti in teoria, hanno una **stabilità numerica** differente, con la forma di Newton che amplifica maggiormente gli errori di arrotondamento per n elevati.

Nella seconda figura, la **forma di Hermite** presenta un andamento simile a quello di Newton. Essendo la loro struttura di base correlata, anche Hermite soffre della stessa **instabilità numerica**, amplificando gli errori al crescere della dimensione del problema.



Esercizio 23. Scrivere una function Matlab,

`x = tridia( b, a, c, y )`

che risolve un sistema tridiagonale, in cui **b**, **a**, **c** sono i vettori contenenti gli elementi sulle 3 diagonali, e **y** è un vettore contenente il termine noto.

Codice MatLab per risolvere un sistema Tridiagonale

```

1 function x = tridia(b,a,c,y)
2
3 %
4 % x = tridia(b,a,c,y)
5 % Input:
6 % a = elementi delle diagonali
7 % b = elementi delle sottodiagonali
8 % c = elementi delle sopradiagonali
9 % y = termini noti
10 %
11 % Output:
12 % x = soluzioni del sistema tridiagonale Ax=y
13 %
14 % Rel. 2-11-2025
15
16
17 n=length(a);
18 if n ~= length(b)+1 || ...
19     n ~= length(c)+1 || ...
20     n ~= length(y)
21     error('dati errati');
22 end
23
24 if a(1)==0
25     error('fattorizzazione indefinita');
26 end
27
28 for i = 1:n-1
29     b(i) = b(i)/a(i);
30     a(i+1) = a(i+1) - b(i)*c(i);
31     if a(i+1)==0
32         error('fattorizzazione indefinita');
33     end
34 end
35
36 x=y(:);
37 for i=2:n
38     x(i)=x(i)-b(i-1)*x(i-1);
39 end
40
41 x(n) = x(n) / a(n);
42 for i = n-1:-1:1
43     x(i) = (x(i) - c(i) * x(i+1)) / a(i);
44 end
45
46 end

```

Esercizio 24. Utilizzando la function del precedente esercizio 23, scrivere la function Matlab,

`y = spline3( xi, fi, x, type )`

in cui (xi, fi) sono i dati di interpolazione e x è un vettore (riga o colonna), contenente i punti in cui valutare la spline cubica interpolante, restituiti nel vettore y, che avrà le stesse dimensioni di x. Infine, type è un flag che vale 1 (valore di default) per la spline cubica not-a-knot, o 0, per quella naturale.

```

1 function val_spline = spline3(ascisse_interpolazione, valori_funzionali, punti_valutazione, tipo)
2 % val_spline = spline3(ascisse_interpolazione, valori_funzionali, punti_valutazione, tipo)
3 % Input:
4 % ascisse_interpolazione : n+1 ascisse di interpolazione
5 % valori_funzionali : n+1 valutazioni funzionali nelle ascisse_interpolazione
6 % punti_valutazione : vettore di punti in cui valutare la spline
7 % tipo : tipo di spline; 0 naturale, 1 not-a-knot
8 % Output:
9 % val_spline : vettore di valori della spline nei punti in punti_valutazione
10 %
11 %
12 %                               Rel. 2-11-2025
13
14     if nargin < 3, error('argomenti insufficienti'), end
15     if nargin == 3, tipo = 1; end
16
17     n_punti = length(ascisse_interpolazione) - 1;
18     if n_punti ~= length(valori_funzionali) - 1 || (tipo ~= 0 && tipo ~= 1), error('dati
19     incorretti'), end
20
21     valori_funzionali = valori_funzionali(:);
22     ascisse_interpolazione = ascisse_interpolazione(:);
23
24     delta_h = ascisse_interpolazione(2:n_punti+1) - ascisse_interpolazione(1:n_punti);
25     phi = delta_h(1:n_punti-1) ./ (delta_h(1:n_punti-1) + delta_h(2:n_punti));
26     Xi = delta_h(2:n_punti) ./ (delta_h(1:n_punti-1) + delta_h(2:n_punti));
27     diff_prime = (valori_funzionali(2:n_punti+1) - valori_funzionali(1:n_punti)) ./ delta_h;
28     diff_seconde = (diff_prime(2:n_punti) - diff_prime(1:n_punti-1)) ./
29     (ascisse_interpolazione(3:n_punti+1) - ascisse_interpolazione(1:n_punti-1));
30
31     m = zeros(n_punti + 1, 1);
32
33     % m trovato con tridia
34     if tipo == 0
35         m(2:n_punti) =
36         tridia(phi(2:n_punti-1), 2*ones((n_punti-1),1), Xi(1:n_punti-2), 6*diff_seconde);
37     else
38         % Codice per spline not-a-knot
39         if length(m) <= 3, error('ascisse insufficienti'); end
40         m(2:n_punti) = tridia([phi(2:n_punti-2); phi(n_punti-1)-Xi(n_punti-1)], ...
41         [2-phi(1); 2*ones(n_punti-3,1); 2-Xi(n_punti-1)], ...
42         [Xi(1)- phi(1); Xi(2:n_punti-2)], ...
43         6*[diff_seconde(1)*Xi(1); diff_seconde(2:n_punti-2);
44         diff_seconde(n_punti-1)*phi(n_punti-1)]);
45         m(1) = 6*diff_seconde(1) - m(2) - m(3);
46         m(n_punti+1) = 6*diff_seconde(n_punti-1) - m(n_punti) - m(n_punti-1);
47     end
48

```

```

45     % calcolo q e r
46     q = diff_prime + (m(1:n_punti) - m(2:n_punti+1)) .* delta_h / 6;
47     r = valori_funzionali(1:n_punti) - (m(1:n_punti) .* (delta_h).^2) / 6;
48
49     % preallocazione per il vettore di output
50     val_spline = zeros(size(punti_valutazione));
51
52     % calcolo spline in punti_valutazione
53     for j = 1:length(punti_valutazione)
54         indice_intervallo = trova_intervallo(punti_valutazione(j), ascisse_interpolazione);
55
56         % Valutazione della spline nel punto j

```

```

57         val_spline(j) = (m(indice_intervallo+1) * (punti_valutazione(j) -
58         ascisse_interpolazione(indice_intervallo))^3 ...
59         + m(indice_intervallo) * (ascisse_interpolazione(indice_intervallo+1) -
60         punti_valutazione(j))^3) / (6 * delta_h(indice_intervallo)) ...
61         + q(indice_intervallo) * (punti_valutazione(j) -
62         ascisse_interpolazione(indice_intervallo)) ...
63         + r(indice_intervallo);
64     end
65
66     return
67 end
68
69 function indice = trova_intervallo(valore_x, ascisse_interp)
70 % indice = trova_intervallo(valore_x, ascisse_interp)
71 % Input:
72 % valore_x : ascissa
73 % ascisse_interp : ascisse di interpolazione
74 % Output:
75 % indice : intervallo in cui si trova valore_x, ritorna -1 se non trovato
76
77 for indice = 1:length(ascisse_interp) - 1
78     if valore_x >= ascisse_interp(indice) && valore_x <= ascisse_interp(indice+1)
79         return
80     end
81 end
82
83 indice = -1; % Se non trova l'intervallo, imposta a -1
84 error("x non fuori dall'intervallo"); % Lancia un errore se x non è nell'intervallo
85 end

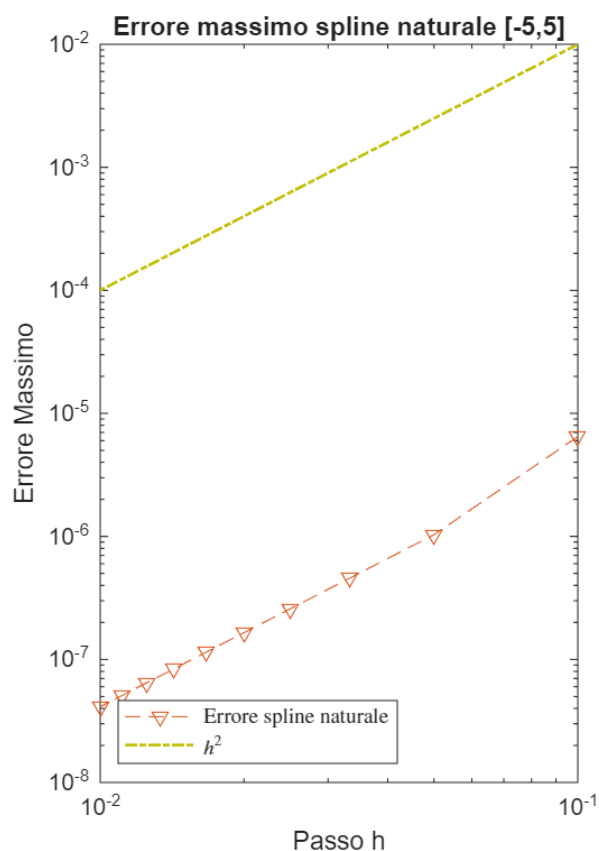
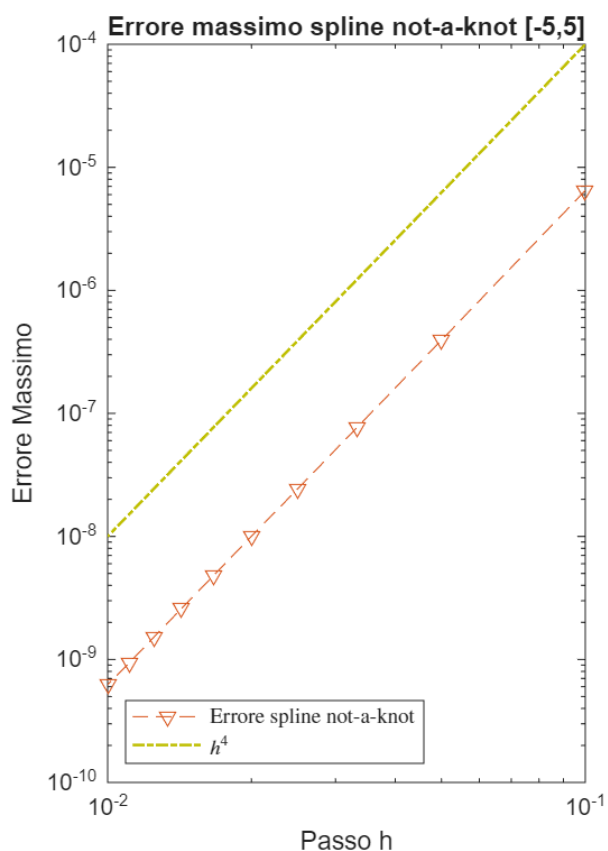
```

Esercizio 25. Utilizzare la function `spline3` per approssimare la funzione di Runge, come nell'esercizio 21, utilizzando $n + 1$ ascisse equidistanti, con $n = 100, 200, \dots, 1000$. Detta $h = 10/n$ la distanza tra due punti consecutivi, graficare, in **loglog** l'errore massimo, calcolato come nell'esercizio 21, rispetto ad h . Con che potenza di h decresce, asintoticamente, l'errore?

Come possiamo osservare nei grafici sotto, abbiamo rappresentato i risultati in una scala logaritmica nel grafico loglog. Questi mostrano come l'errore diminuisce al diminuire di $h = 10/n$.

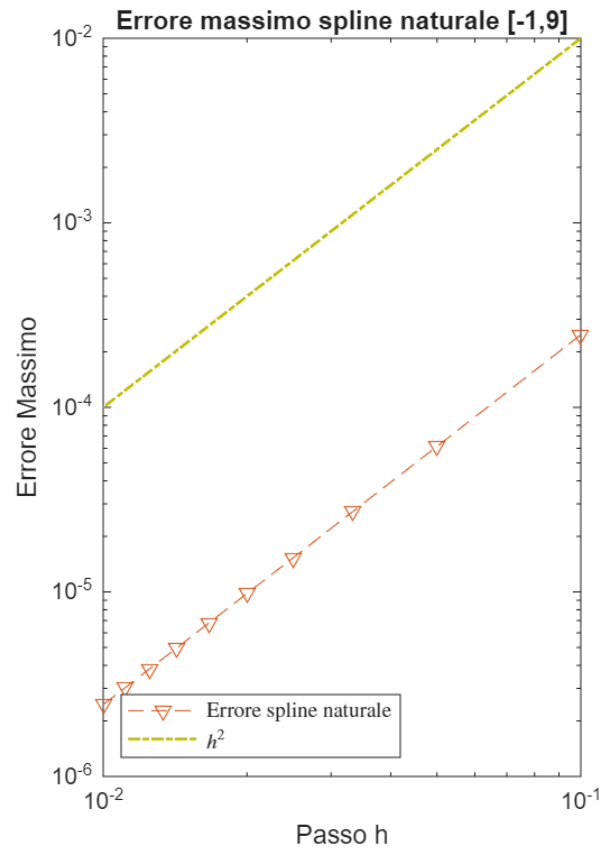
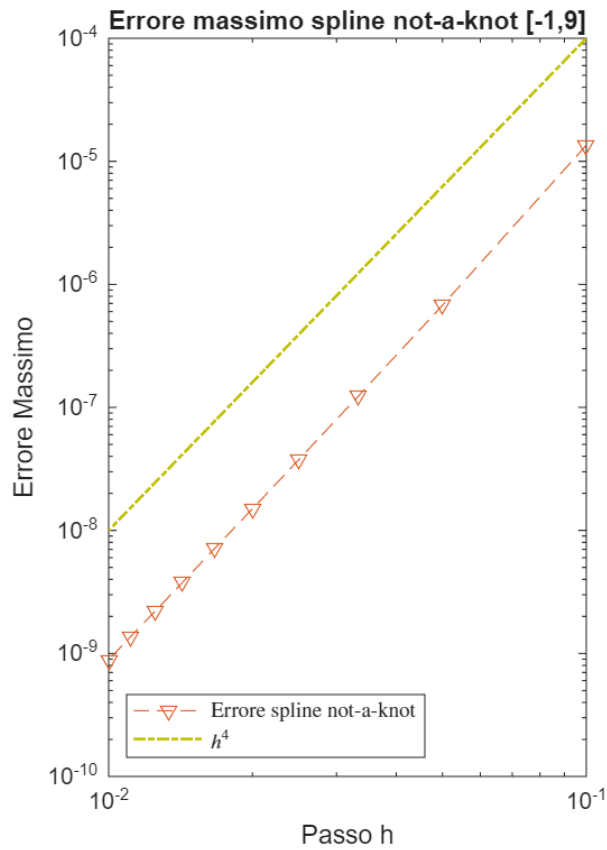
Questo fenomeno, però avviene a velocità diverse. Per la spline naturale, l'errore diminuisce più lentamente: circa come h^2 . Mentre, con la spline not-a-knot, l'errore scende più velocemente: circa come h^4 .

Ciò ci fa notare che, anche se le spline utilizzano gli stessi punti, la scelta delle condizioni dei bordi cambia molto le precisioni, infatti, la spline not-a-knot impone condizioni più regolari alle estremità



Esercizio 26. Ripetere il precedente esercizio considerando, invece dell'intervallo $[-5, 5]$, l'intervallo $[-1, 9]$. Commentare i risultati ottenuti, rispetto a quelli del precedente esercizio.

Il grafico sottostante ci mostra l'andamento dell'errore. Al contrario del precedente intervallo, gli errori iniziali sono più grandi, al contrario l'andamento asintotico rimane uguale.

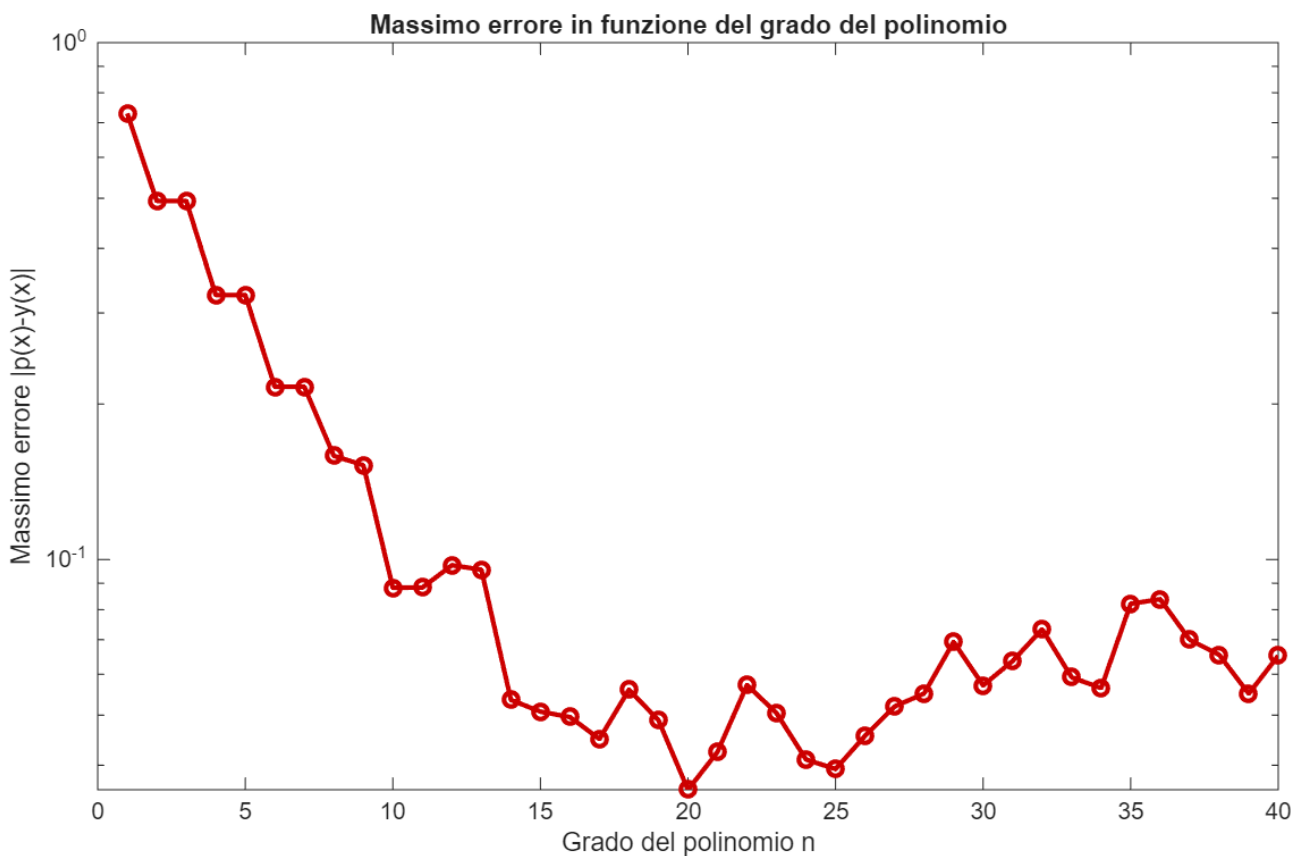


Esercizio 27. La function `polyfit` calcola il polinomio interpolante ai minimi quadrati. Creare dei dati esatti e perturbati, relativi alla funzione di Runge, con le seguenti istruzioni:

```
x = linspace(-5,5,1001);
y = 1./(1+x.^2);
rng(0)
r = ( rand(1,1001)-rand(1,1001) )/5;
yr = y + r;
plot(x,yr,'r.',x,y,'b','LineWidth',2)
xlabel('x'), ylabel('y')
shg
```

Calcolare il massimo errore tra il valore restituito dal polinomio di approssimazione ai minimi quadrati di grado n , costruito sulle coppie di dati (x, yr) , rispetto al valore esatto contenuto nel vettore y . Graficare l'errore rispetto a n , per $n = 1, 2, \dots, 40$, in formato **semilogy**. Commentare i risultati ottenuti.

Il grafico sottostante ci mostra che al crescere di n , l'errore massimo dell'interpolazione tramite `polyfit` mostra un andamento prevalentemente decrescente. Si osserva come con n compreso tra 0 e 10 l'andamento sia decisamente decrescente, mentre, con $n > 10$ ci siano più oscillazioni, nonostante ciò, il valore dell'errore rimane minore di 10^{-1} .



Esercizio 28. Costruire una function Matlab, `ci = ncotes( n )` che calcoli i coefficienti della formula di Newton-Cotes di grado n . Costruire una tabella con tutti i coefficienti delle formule ammissibili per ragioni di condizionamento. I coefficienti della tabella devono essere riportati come numeri razionali.

```
1 function coefficiente_interpolazione = ncotes(grado)
2
3 % coefficiente_interpolazione = ncotes(grado)
4 % Calcola i coefficienti della formula di N-C di grado 'grado'
5 % Rel. 2-11-2025
6
7 if nargin < 1, error('input errato'), end
8
9 coefficiente_interpolazione = zeros(grado + 1, 1);
10
11 for indice = 0 : ceil(grado / 2)
12     coefficiente_interpolazione(indice + 1) = calcola_coefficiente_i(indice, grado);
13     coefficiente_interpolazione(grado + 1 - indice) = coefficiente_interpolazione(indice +
14 1);
15 end
16 return
17 end
18
19 function coeff_i = calcola_coefficiente_i(i, grado)
20
21 vettore_nn = [0 : i - 1, i + 1 : grado];
22 fattore_scala = prod(i - vettore_nn);
23 polinomio_a = poly(vettore_nn);
24 denominatori_integrazione = (grado + 1) : -1 : 1;
25 polinomio_a_integrato = [polinomio_a ./ denominatori_integrazione, 0];
26 coeff_i = polyval(polinomio_a_integrato, grado) / fattore_scala;
27
28 return
29 end
```

Nella tabella sottostante si possono vedere i risultati di $n_cotes(n)$ con $n = 1, \dots, 7, 9$.

n = 1	n = 2	n = 3	n = 4	n = 5	n = 6	n = 7	n = 9
$\frac{1}{2}$	$\frac{1}{3}$	$\frac{3}{8}$	$\frac{14}{45}$	$\frac{95}{288}$	$\frac{41}{140}$	$\frac{108}{355}$	$\frac{103}{453}$
$\frac{1}{2}$	$\frac{4}{3}$	$\frac{9}{8}$	$\frac{64}{45}$	$\frac{125}{96}$	$\frac{54}{35}$	$\frac{810}{559}$	$\frac{419}{265}$
	$\frac{1}{3}$	$\frac{9}{8}$	$\frac{8}{15}$	$\frac{125}{144}$	$\frac{27}{140}$	$\frac{343}{640}$	$\frac{23}{212}$
		$\frac{3}{8}$	$\frac{64}{45}$	$\frac{125}{144}$	$\frac{68}{35}$	$\frac{649}{536}$	$\frac{307}{158}$
			$\frac{14}{45}$	$\frac{125}{96}$	$\frac{27}{140}$	$\frac{649}{536}$	$\frac{213}{367}$
				$\frac{95}{288}$	$\frac{54}{35}$	$\frac{343}{640}$	$\frac{213}{367}$
					$\frac{41}{140}$	$\frac{810}{559}$	$\frac{307}{158}$
						$\frac{108}{355}$	$\frac{23}{212}$
							$\frac{419}{265}$
							$\frac{103}{453}$

Esercizio 29. Costruire una function Matlab, `Ikn = ncotescomp( fx, a, b, k, n )` che calcoli l'approssimazione dell'integrale della funzione `fx` sull'intervallo `[a,b]`, utilizzando la formula di Newton-Cotes di grado `k` su `n + 1` ascisse equidistanti.

```

1 function integrale_approssimato = ncotescomp(funzione_f, limite_a, limite_b, grado_k,
  numero_intervalli)
2 % integrale_approssimato = ncotes_composita(funzione_f, limite_a, limite_b, grado_k,
  numero_intervalli)
3 % Input:
4 %   funzione_f      : funzione da integrare (handle della funzione)
5 %   limite_a, limite_b : estremi di integrazione [a,b]
6 %   grado_k         : grado della formula di Newton-Cotes (k)
7 %   numero_intervalli : numero totale di sottointervalli (n)
8 % Output:
9 %   integrale_approssimato : approssimazione dell'integrale di funzione_f in [a,b]
10 %
11 % Rel. 2-11-2025
12
13 if nargin < 5, error('dati insufficienti'), end
14
15 if limite_b <= limite_a || mod(numero_intervalli, grado_k) ~= 0 || numero_intervalli < 1
16     error('dati non validi: controllare estremi, n o k');
17 end
18
19 if grado_k < 1 || grado_k == 8 || grado_k > 9
20     error('problema malcondizionato: k deve essere tra 1 e 9, escluso 8');
21 end
22
23 passo_h = (limite_b - limite_a) / numero_intervalli;
24
25
26 punti_x = (limite_a : passo_h : limite_b)';
27 valori_f = funzione_f(punti_x);
28 coefficienti_nc = ncotes(grado_k);
29 integrale_approssimato = 0;
30
31 for indice_i = 1 : numero_intervalli / grado_k
32     indici_correnti = ((indice_i - 1) * grado_k : indice_i * grado_k) + 1;
33     valori_sottointervallo = valori_f(indici_correnti);
34
35     integrale_approssimato = integrale_approssimato + sum(coefficienti_nc .*
  valori_sottointervallo);
36 end
37
38 integrale_approssimato = integrale_approssimato * passo_h;
39
40 return
41 end

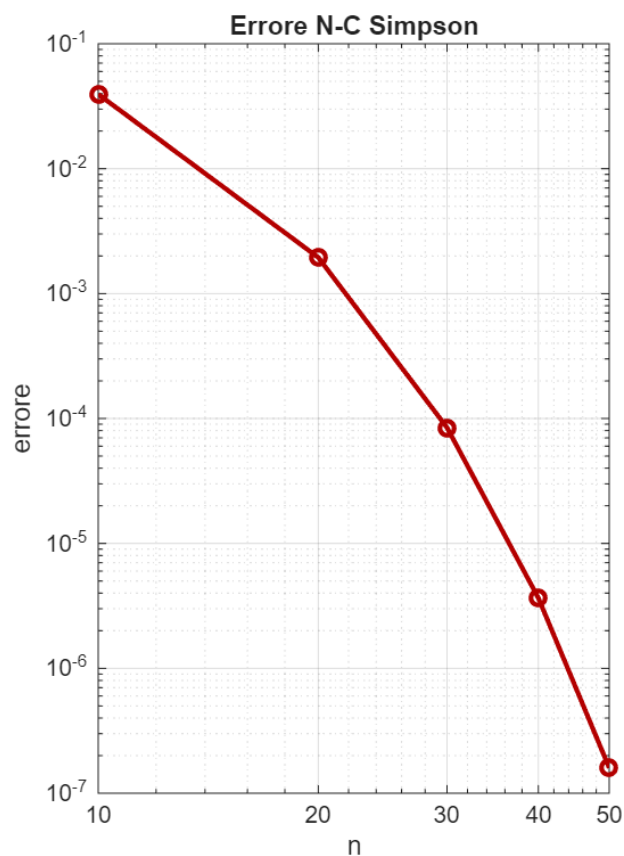
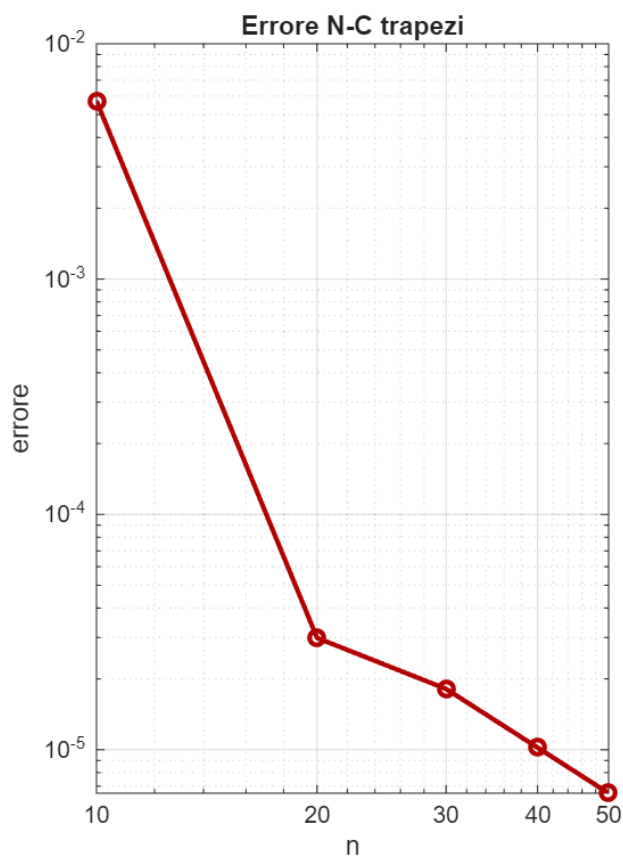
```

Esercizio 30. Utilizzare la function del precedente esercizio per approssimare

$$\int_0^{10} \frac{1}{1+x^2} dx \equiv \arctan(10),$$

con le formule composite dei trapezi e di Simpson. Tabulare l'errore per $n = 10, 20, 30, 40, 50$.

Nel grafico sottostante possiamo confrontare l'errore della formula composta dei trapezi e della formula di Simpson.



Di seguito vengono riportati i valori degli errori delle due formule per $n = 10, 20, 30, 40, 50$.

	10	20	30	40	50
Trapezi	5.71×10^{-3}	2.99×10^{-5}	1.81×10^{-5}	1.02×10^{-5}	6.53×10^{-6}
Simpson	3.95×10^{-2}	1.94×10^{-3}	8.45×10^{-5}	3.66×10^{-6}	1.60×10^{-7}